Reviewer Pack — Minimal Hodge Cohomology and Circuit Monotone Width Inequali…

Entry 57d67ee73362 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 19:49:38 UTC

Minimal Hodge Cohomology and Circuit Monotone Width Inequality

Entry ID: `57d67ee73362` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 19:49:38 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every d-regular graph G, the minimal Hodge cohomology dimension h(G) of its associated algebraic variety is linearly correlated with its circuit monotone width w(G), such that h(G) = Θ(w(G))

Rationale (proposer's reasoning):

> The minimal Hodge cohomology dimension of a variety could provide a geometric complexity measure, which might reveal hidden structures related to circuit complexity. A correlation between these dimensions and circuit monotone width could suggest a deeper connection between algebraic geometry and computational complexity.

Taxonomy category: `geometric_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `04df9123541c8e4f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a d-regular graph G with n vertices, if the correlation coefficient r(h(G), w(G)) is greater than or equal to 0.7 and all computed values of h(G) are within the range [1.2 * w(G), 0.8 * w(G)] for at least 24 out of 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'Hodge Theory' AND 'circuit monotone width'` - `'minimal Hodge cohomology' AND 'Boolean circuit complexity'` - `'algebraic variety' AND 'graph circuit monotone width'`

Top relevant hits considered: - [http://arxiv.org/abs/1801.10489v3] Hodge level for weighted complete intersections - [http://arxiv.org/abs/2102.03481v3] Noncommutative Hodge conjecture - [http://arxiv.org/abs/2211.11405v4] On a Hodge locus - [http://arxiv.org/abs/1901.05780v2] Hodge filtration, minimal exponent, and local vanishing - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [http://arxiv.org/abs/0912.2255v3] Test ideals via algebras of p^{-e}-linear maps - [http://arxiv.org/abs/1812.11710 Geometric Satake correspondence for affine Kac-Moody Lie algebras of type A - [s2:10.1007/978-3-030-58150-3_40] Succinct Monotone Circuit Certification: Planarity and Parameterized Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for j in range(n):
            i_max = rank
            for i in range(rank, m):
                if abs(A[i][j]) > abs(A[i_max][j]):
                    i_max = i
            if A[i_max][j] == 0:
                continue
            A[rank], A[i_max] = A[i_max], A[rank]
            for i in range(m):
                if i != rank:
                    factor = -A[i][j] / A[rank][j]
                    for k in range(n):
                        A[i][k] += factor * A[rank][k]
            rank += 1
        return rank
```

```
def min_hodge_cohomology(G):
    n = len(G)
    adj_matrix = [[0] * n for _ in range(n)]
    for u, v in G:
        adj_matrix[u][v] = 1
        adj_matrix[v][u] = 1

    laplacian = [[0] * n for _ in range(n)]
    for i in range(n):
        degree = sum(adj_matrix[i][j] for j in range(n))
        laplacian[i][i] = degree
        for j in range(i + 1, n):
            laplacian[i][j] = -adj_matrix[i][j]
            laplacian[j][i] = -adj_matrix[i][j]

    return gaussian_elimination(laplacian)

def circuit_monotone_width(G):
    n = len(G)
    max_width = 0
    for i in range(n):
        width = 1
        visited = [False] * n
        stack = [i]
        while stack:
            u = stack.pop()
            if not visited[u]:
                visited[u] = True
                for v in G[u]:
                    if not visited[v]:
                        stack.append(v)
                        width += 1
        max_width = max(max_width, width)
    return max_width

def generate_d_regular_graph(n, d):
    if (n * d) % 2 != 0:
        raise ValueError("Graph size must be a multiple of the degree")

    G = [[] for _ in range(n)]
    edges_added = set()
    for u in range(n):
        for v in range(u + 1, n):
            if len(G[u]) < d and len(G[v]) < d:
                if (u, v) not in edges_added and (v, u) not in edges_added:
                    G[u].append(v)
                    G[v].append(u)
                    edges_added.add((u, v))

    return G

n_values = [5, 10, 15, 20, 30, 40]
h_values = []
w_values = []

for n in n_values:
```

```
        for _ in range(5):
            d = random.randint(2, n - 1)
            G = generate_d_regular_graph(n, d)
            h = min_hodge_cohomology(G)
            w = circuit_monotone_width(G)
            h_values.append(h)
            w_values.append(w)

    if len(h_values) < 30:
        return {
            "metric_name": "Minimal Hodge Cohomology and Circuit Monotone Width",
            "metric_value": None,
            "instances_tested": len(h_values),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "Insufficient instances tested"
        }

    h_avg = sum(h_values) / len(h_values)
    w_avg = sum(w_values) / len(w_values)
    correlation_coefficient = sum((h - h_avg) * (w - w_avg) for h, w in zip(h_values, w_values)) /

    if correlation_coefficient >= 0.7 and all(1.2 * w <= h <= 0.8 * w for h, w in zip(h_values, w_
        return {
            "metric_name": "Minimal Hodge Cohomology and Circuit Monotone Width",
            "metric_value": correlation_coefficient,
            "instances_tested": len(h_values),
            "n_max": max(n_values),
            "conjecture_holds": True,
            "counterexample": ""
        }
    else:
        return {
            "metric_name": "Minimal Hodge Cohomology and Circuit Monotone Width",
            "metric_value": correlation_coefficient,
            "instances_tested": len(h_values),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": f"Correlation: {correlation_coefficient}, H values out of range"
        }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
```

```
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Correlation out of range\" first_failing_seed={
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5917219a.py", line 142, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5917219a.py", line 98, in run_trial
    G = generate_d_regular_graph(n, d)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5917219a.py", line 77, in generate_d_re
    raise ValueError("Graph size must be a multiple of the degree")
ValueError: Graph size must be a multiple of the degree
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete the required computations to evaluate the conjecture. | next: Review and fix the error in the test code that caused it to crash, then rerun the test with a valid graph size that is a multiple of the degree.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15494
2	preregistration	ollama_remote	glm4:latest	0	0	15715
3	novelty	ollama_remote	glm4:latest	0	0	10894
4	novelty	ollama_remote	glm4:latest	0	0	11675
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21722
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13846
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13816
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17528

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	17418

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 138108 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/57d67ee73362.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/57d67ee73362.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/57d67ee73362.tar.gz` (if generated)